



universidad
de león



Escuela de Ingenierías Industrial, Informática y Aeroespacial

GRADO EN INGENIERÍA DE DATOS E INTELIGENCIA ARTIFICIAL

Trabajo de Fin de Grado

Sistemas de Movilidad Autónoma en Simuladores Urbanos mediante Redes Neuronales y Algoritmos Evolutivos

Autonomous Mobility Systems in Urban Simulators Using Neural Networks and
Evolutionary Algorithms

Autor: Sergio Alves Martín
Tutor/es: Sergio Rubio Martín / Alicia Merayo Corcoba

(Junio, 2026)

UNIVERSIDAD DE LEÓN
Escuela de Ingenierías Industrial, Informática y
Aeroespacial

GRADO EN INGENIERÍA DE DATOS E
INTELIGENCIA ARTIFICIAL
Trabajo de Fin de Grado

ALUMNO: Sergio Alves Martín

TUTOR/ES: Sergio Rubio Martín / Alicia Merayo Corcoba

TÍTULO: Sistemas de Movilidad Autónoma en Simuladores Urbanos mediante Redes Neuronales y Algoritmos Evolutivos

TITLE: Autonomous Mobility Systems in Urban Simulators Using Neural Networks and Evolutionary Algorithms

CONVOCATORIA: Junio, 2026

RESUMEN:

El trabajo desarrolla un sistema de movilidad autónoma para *CognitiveCarSimulatorReloaded*, simulador urbano en Unreal Engine 5 orientado a rehabilitación y valoración cognitiva con ASPAYM. La solución incorpora vehículos con sensores simulados, interpretación de señales y un controlador neuronal entrenado con algoritmos evolutivos, junto con persistencia de modelos y análisis de métricas CSV. La evaluación de referencia incluye 2067 generaciones de entrenamiento y seis sesiones de test con 1560 evaluaciones: fitness máximo de 112447.60 en entrenamiento, 85530.55 en test y una tasa agregada de acierto del 25.19 %.

ABSTRACT:

This thesis develops an autonomous mobility system for *CognitiveCarSimulatorReloaded*, an Unreal Engine 5 urban simulator aimed at cognitive rehabilitation and assessment with ASPAYM. The solution adds vehicles with simulated sensors, traffic-sign interpretation and a neural controller trained through evolutionary algorithms, plus model persistence and CSV metric analysis. The reference evaluation comprises 2067 training generations and six test sessions with 1560 evaluations: maximum fitness values of 112447.60 in training and 85530.55 in testing, with an aggregate success rate of 25.19 %.

Palabras clave: simulación urbana, movilidad autónoma, redes neuronales, algoritmos evolutivos, Unreal Engine.

Firma del alumno:

VºBº Tutor/es:

Resumen

CognitiveCarSimulatorReloaded es un simulador urbano desarrollado en Unreal Engine 5 para apoyar actividades de rehabilitación y valoración cognitiva relacionadas con la conducción. Este Trabajo de Fin de Grado aborda una de las carencias detectadas en la versión previa del sistema: la ausencia de tráfico autónomo capaz de generar situaciones dinámicas y repetibles. El objetivo es diseñar e integrar una arquitectura de movilidad vehicular que perciba el entorno, actúe sobre un vehículo con físicas reales, aprenda mediante experimentación y produzca información suficiente para evaluar su comportamiento. La solución combina once sensores distribuidos en un campo de visión de 180 grados, variables dinámicas del vehículo y contexto de semáforos, stops y cedas. Estas entradas alimentan un perceptrón multicapa ligero cuya configuración actual contiene 22 entradas, una capa oculta de 16 neuronas y dos salidas continuas para dirección y aceleración o frenado. La política de control se optimiza mediante un algoritmo evolutivo que genera poblaciones, conserva modelos, muta pesos y selecciona individuos según una función de aptitud instrumentada. El sistema incluye además persistencia de modelos, separación entre puntos de entrenamiento y test, selección de trayectorias en intersecciones y actores específicos para validar el cumplimiento de señales. Para facilitar la trazabilidad, cada ejecución exporta métricas detalladas a CSV y se analiza mediante scripts de Python que comprueban la calidad de los datos y generan informes y visualizaciones. La evaluación de referencia utiliza una sesión de entrenamiento de 2067 generaciones y 66144 registros, junto con seis sesiones de test que suman 1560 evaluaciones sobre 52 puntos de aparición. Se alcanzan valores máximos de fitness de 112447.60 en entrenamiento y 85530.55 en test. La tasa agregada de acierto es del 25.19 %, con un intervalo de confianza del 95 % entre el 23.10 % y el 27.41 %. Frente a los bloques de test de los días 2 y 5 de junio, aumentan la tasa de acierto, el fitness medio y el tiempo de vida. El entrenamiento elimina el estancamiento LAZY y reduce los fallos legales, aunque no presenta convergencia clara y mantiene posibles atascos ante señales. Las colisiones en intersecciones y la variabilidad entre puntos de aparición continúan siendo las principales limitaciones. Por ello, el trabajo se presenta como una base funcional y medible sobre la que seguir mejorando percepción, entrenamiento dirigido, interacción entre vehículos y validación en escenarios de uso más amplios.

Abstract

CognitiveCarSimulatorReloaded is an urban driving simulator developed in Unreal Engine 5 to support cognitive rehabilitation and assessment activities related to driving. This Bachelor's Thesis addresses one of the main limitations identified in the previous version of the system: the lack of autonomous traffic capable of producing dynamic and repeatable situations. The objective is to design and integrate a vehicle mobility architecture that perceives its surroundings, controls a physically simulated car, learns through experimentation and produces enough information to evaluate its behaviour. The proposed solution combines eleven sensors distributed across a 180-degree field of view, dynamic vehicle variables and contextual information from traffic lights, stop signs and yield signs. These inputs feed a lightweight multilayer perceptron whose current configuration contains 22 inputs, one hidden layer with 16 neurons and two continuous outputs for steering and acceleration or braking. The control policy is optimised through an evolutionary algorithm that creates populations, preserves models, mutates weights and selects individuals according to an instrumented fitness function. The system also provides model persistence, separate training and test spawn points, route selection at intersections and dedicated actors for validating compliance with traffic controls. To support traceability, every run exports detailed metrics to CSV files, which are processed by Python scripts that validate data quality and generate reports and visualisations. The reference evaluation comprises a training session with 2067 generations and 66144 vehicle records, together with six test sessions totalling 1560 evaluations across 52 spawn points. These experiments achieve maximum fitness values of 112447.60 during training and 85530.55 during testing. The aggregate success rate is 25.19 %, with a 95 % confidence interval from 23.10 % to 27.41 %. Compared with the test blocks run on 2 and 5 June, success rate, mean fitness and survival time all increase. Training removes LAZY stagnation and reduces legal failures, although it does not show clear convergence and still presents possible traffic-control traps. Collisions at intersections and performance variability between spawn points remain the main limitations. The work should therefore be understood as a functional and measurable baseline for further improvements in perception, targeted training, interaction between vehicles and validation under broader usage scenarios.

Índice general

Ficha del trabajo	I
Resumen	II
Abstract	III
Glosario	VIII
1. Estudio del problema	1
1.1. Contexto	1
1.2. Problema que se resuelve	2
1.3. Objetivo general	2
1.4. Objetivos específicos	3
1.5. Estructura del trabajo	3
2. Estado del arte	4
2.1. Simuladores de conducción	4
2.2. Agentes autónomos en entornos urbanos	4
2.3. Redes neuronales y neuroevolución	5
2.4. Retos específicos	6
3. Tecnologías y herramientas	7
3.1. Unreal Engine 5	7
3.2. Sistema de físicas Chaos Vehicle	7
3.3. Raycasting y percepción local	8
3.4. Red neuronal	8
3.5. Algoritmos evolutivos	9
3.6. Análisis de datos	9
3.7. Control de versiones	9
4. Gestión del proyecto	10
4.1. Metodología	10
4.2. Cronología de decisiones	10
4.3. Alcance	11
4.4. Gestión de riesgos	12
4.5. Estimación de recursos y presupuesto	13
5. Núcleo del trabajo	14

5.1. Descripción general de la solución	14
5.2. Subsistemas	14
5.2.1. Gestor evolutivo	14
5.2.2. Controlador de IA y red neuronal	15
5.2.3. Percepción mediante sensores	15
5.2.4. Sistema de fitness	16
5.2.5. Semáforos	16
5.2.6. Stop y ceda el paso	17
5.3. Pipeline experimental	17
6. Resultados	18
6.1. Datos analizados	18
6.2. Resumen cuantitativo	18
6.3. Evolución del entrenamiento	19
6.4. Evaluación de test	20
6.5. Razones de finalización	20
6.6. Comportamiento ante señalización	21
6.7. Dificultad por puntos de aparición	21
6.8. Conclusión experimental	21
7. Normativa y accesibilidad	22
7.1. Protección de datos	22
7.2. Seguridad y uso responsable	22
7.3. Accesibilidad	23
8. Conclusiones y líneas futuras	24
8.1. Conclusiones	24
8.2. Aportaciones realizadas	25
8.3. Problemas encontrados	25
8.4. Líneas futuras	26
Agradecimientos	27
Bibliografía	28
A. Fuentes analizadas	29
A.1. Documentación administrativa y de formato	29
A.2. Actas de reunión	29
A.3. Descripciones del programa	30
A.4. Resultados experimentales	30

Índice de figuras

- 6.1. Tendencia de fitness y tiempo medio en el entrenamiento del 8 de junio de 2026. Fuente: análisis generado a partir de los CSV del proyecto. . . . 19

Índice de tablas

4.1. Principales hitos y decisiones documentadas en reuniones. Fuente: elaboración propia a partir de las actas de seguimiento.	10
4.2. Riesgos principales del proyecto. Fuente: elaboración propia.	12
6.1. Resultados principales del entrenamiento y del bloque de test del 8 de junio de 2026. Fuente: elaboración propia a partir de los CSV del proyecto.	18
6.2. Evolución de los bloques de test disponibles en junio de 2026. Fuente: elaboración propia a partir de Fitness_Debug.csv.	20

Glosario

ASPAYM	Asociación/Fundación colaboradora orientada a mejorar la calidad de vida de personas con lesión medular, gran discapacidad física y daño cerebral adquirido.
Blueprint	Sistema visual de programación de Unreal Engine empleado para implementar lógica de juego, interacción, controladores y sistemas de simulación.
Chaos Vehicle	Sistema de físicas de Unreal Engine 5 utilizado para simular el comportamiento dinámico del vehículo.
DCA	Daño Cerebral Adquirido.
Fitness	Función de aptitud que cuantifica el comportamiento de cada vehículo autónomo durante una generación evolutiva.
NavMesh	Malla de navegación utilizada por motores de videojuegos para calcular rutas transitables. En este proyecto fue estudiada y posteriormente descartada como solución principal para vehículos.
NPC	<i>Non-Player Character</i> ; agente controlado por el sistema y no por el usuario.
Raycasting	Técnica de consulta geométrica que lanza rayos o trazas desde un actor para detectar obstáculos y distancias en el entorno.
RN	Red neuronal artificial.
UE5	Unreal Engine 5.

Capítulo 1

Estudio del problema

1.1 CONTEXTO

La conducción es una actividad compleja que exige coordinación motora, atención sostenida, atención dividida, interpretación de señales, percepción espacial y toma de decisiones bajo restricciones temporales. En personas que han sufrido daño cerebral adquirido, estas capacidades pueden verse alteradas aunque existan adaptaciones físicas que permitan manejar un vehículo. Por ello, antes de exponer al paciente a situaciones reales de tráfico, resulta de interés disponer de entornos controlados donde observar, entrenar y valorar comportamientos de conducción de forma segura.

El proyecto *CognitiveCarSimulatorReloaded* nace dentro de esta necesidad. La propuesta original define un simulador realista de conducción urbana para entrenamiento y valoración de alteraciones cognitivas tras un daño cerebral adquirido, desarrollado en colaboración con ASPAYM [1]. La versión inicial del simulador, desarrollada durante el curso 2024-2025, ya incluía un entorno urbano, vehículo, interacción física, señalización y registro de datos [2]. Sin embargo, las evaluaciones posteriores identificaron una carencia clave: el escenario necesitaba tráfico autónomo y dinámico para trasladar al usuario a una situación más realista.

El presente TFG aborda esa carencia desde el ámbito de la Ingeniería de Datos e Inteligencia Artificial. El foco no se sitúa en crear un simulador desde cero, sino en diseñar e integrar modelos de movilidad autónoma capaces de aprender comportamientos de conducción dentro de un entorno urbano ya existente. El trabajo combina redes neuronales, algoritmos evolutivos, sensores simulados, control físico en tiempo real y análisis de datos experimentales.

1.2 PROBLEMA QUE SE RESUELVE

El problema técnico principal consiste en conseguir que agentes autónomos, inicialmente vehículos, circulen por un mapa urbano de forma suficientemente estable, medible y compatible con las normas básicas de tráfico. A diferencia de un recorrido prefijado, un agente autónomo debe responder a múltiples condiciones locales: límites de la calzada, obstáculos, intersecciones, semáforos, señales de stop, señales de ceda el paso y cambios en su propio estado dinámico.

Durante el desarrollo se evaluaron soluciones deterministas de navegación, como el uso de NavMesh o recorridos mediante *Path Finding*. Estas aproximaciones resultaron útiles como punto de partida, pero presentaron limitaciones para el objetivo clínico y experimental del simulador: rutas rígidas, giros poco naturales, dificultad para representar la toma de decisiones local y menor capacidad de adaptación a situaciones imprevistas. Como consecuencia, el proyecto evolucionó hacia una solución basada en percepción local mediante sensores y aprendizaje evolutivo.

Además, el problema requiere que el comportamiento obtenido sea observable y explicable. No basta con que los vehículos se desplacen por el escenario; es necesario registrar por qué avanzan, se detienen, colisionan, incumplen una señal o completan correctamente una maniobra. Esta trazabilidad condiciona la arquitectura de la solución, porque cada decisión de control debe poder relacionarse con sensores, contexto de tráfico, componentes de fitness y resultados experimentales.

1.3 OBJETIVO GENERAL

El objetivo general del trabajo es desarrollar e integrar un sistema de movilidad autónoma para un simulador urbano de conducción, basado en redes neuronales y algoritmos evolutivos, que permita entrenar, evaluar y analizar vehículos autónomos dentro de Unreal Engine 5.

De forma complementaria, el proyecto busca dejar una base reutilizable para futuras ampliaciones del simulador. La movilidad vehicular autónoma actúa como primer bloque técnico sobre el que podrán incorporarse perfiles de conducción, otros agentes dinámicos y escenarios de validación más cercanos al uso clínico previsto.

1.4 OBJETIVOS ESPECÍFICOS

Los objetivos específicos son:

- Analizar el estado inicial del simulador y las restricciones técnicas heredadas de la versión previa.
- Diseñar una arquitectura de percepción para vehículos autónomos basada en sensores simulados mediante raycasting.
- Implementar una red neuronal ligera capaz de transformar entradas sensoriales y de contexto en acciones de conducción.
- Diseñar un proceso de entrenamiento evolutivo que evalúe poblaciones de vehículos, seleccione individuos destacados y aplique mutaciones sobre sus pesos.
- Integrar señales de tráfico, semáforos, stops y cedas en la lógica de percepción, parada y validación de la IA.
- Construir una función de fitness interpretable, con componentes positivos y penalizaciones, que permita medir progreso, errores y comportamientos no deseados.
- Generar registros experimentales en CSV y analizarlos mediante scripts reproducibles para evaluar entrenamiento y test.
- Identificar limitaciones, riesgos y líneas futuras para ampliar la movilidad autónoma a perfiles de conductor, peatones, bicicletas y otros agentes.

1.5 ESTRUCTURA DEL TRABAJO

El documento se organiza en ocho capítulos principales. Tras este estudio del problema, el capítulo 2 resume el estado del arte y los retos asociados a simuladores, agentes autónomos y aprendizaje evolutivo. El capítulo 3 describe las tecnologías y herramientas utilizadas. El capítulo 4 presenta la gestión del proyecto, la evolución temporal, los riesgos y una estimación preliminar de recursos. El capítulo 5 desarrolla el núcleo técnico de la solución. El capítulo 6 recoge los resultados experimentales. El capítulo 7 trata normativa, protección de datos y accesibilidad. Finalmente, el capítulo 8 expone conclusiones, aportaciones y líneas futuras.

Capítulo 2

Estado del arte

2.1 SIMULADORES DE CONDUCCIÓN

Los simuladores de conducción permiten reproducir situaciones de tráfico en un entorno controlado, repetible y seguro. En el ámbito formativo se utilizan para entrenar maniobras, respuesta ante eventos inesperados y familiarización con normas de circulación. En el ámbito sanitario, su valor reside en que permiten observar capacidades cognitivas y motoras sin exponer al usuario a riesgos reales. La versión previa de *CognitiveCarSimulator* ya se apoyaba en esta idea: un entorno urbano en Unreal Engine donde registrar tiempos de reacción, velocidades, colisiones y datos exportables para especialistas [2].

Para que un simulador de rehabilitación resulte convincente, el entorno no puede limitarse a una vía vacía. El usuario debe compartir la escena con otros agentes que creen carga cognitiva: vehículos que se detienen, peatones que cruzan, ciclistas, señales y elementos dinámicos. La propuesta de *CognitiveCarSimulatorReloaded* identifica precisamente esa necesidad al plantear IAs conductoras configurables, peatones, bicicletas y animales como ampliaciones progresivas del simulador [1].

2.2 AGENTES AUTÓNOMOS EN ENTORNOS URBANOS

La navegación autónoma en videojuegos y simulación suele abordarse mediante técnicas deterministas, como grafos de rutas, splines o mallas de navegación. Estas técnicas son robustas cuando el objetivo es desplazar agentes por zonas transitables con trayectorias controladas, pero pueden producir comportamientos rígidos si se aplican directamente a vehículos físicos. Un coche no se desplaza como un personaje humano: tiene inercia, radio de giro, aceleración, frenada, fricción y restricciones de control continuas.

En este proyecto se comprobó esa diferencia durante las primeras iteraciones. El uso de NavMesh y Path Finding fue estudiado para permitir recorridos por la ciudad, pero las reuniones de seguimiento y los informes técnicos documentan problemas de rigidez, dificultad en intersecciones y falta de naturalidad. Por ello se adoptó una estrategia más cercana a la percepción local: el agente no sigue una ruta global cerrada, sino que interpreta distancias, estado del tráfico y contexto normativo para producir acciones continuas.

2.3 REDES NEURONALES Y NEUROEVOLUCIÓN

Las redes neuronales artificiales permiten aproximar funciones complejas a partir de entradas numéricas. En este TFG se emplean como política de control: reciben sensores, velocidad, contexto de señalización y estado de parada, y producen dirección y aceleración/frenado. La configuración vigente es deliberadamente ligera para poder ejecutarse en tiempo real sobre múltiples vehículos: 22 entradas, una capa oculta de 16 neuronas y dos salidas continuas con activación hiperbólica.

El entrenamiento no se plantea como aprendizaje supervisado, ya que no se dispone de un conjunto amplio de trayectorias humanas etiquetadas dentro del simulador. En su lugar, se adopta un algoritmo evolutivo. Cada individuo de una población representa un conjunto de pesos de la red neuronal. La simulación evalúa el comportamiento mediante fitness; los mejores pesos se conservan, se mutan y se reutilizan en generaciones posteriores. Este enfoque resulta adecuado para explorar comportamientos en entornos donde la señal de aprendizaje procede de la interacción con el mundo simulado.

La elección de un enfoque evolutivo también facilita trabajar con objetivos parcialmente contradictorios. Un vehículo debe avanzar, pero no a cualquier precio; debe mantener velocidad útil, pero frenar ante obstáculos; debe explorar la ciudad, pero respetar stops, cedas y semáforos. En este contexto, el fitness actúa como mecanismo de equilibrio entre progreso, seguridad, cumplimiento normativo y estabilidad de la conducción.

2.4 RETOS ESPECÍFICOS

Los principales retos identificados son:

- **Simulación física:** el control neuronal debe traducirse a entradas de un vehículo con físicas reales, evitando aceleraciones, frenadas o giros no plausibles.
- **Intersecciones:** los cruces concentran la mayor complejidad, porque combinan decisión de trayectoria, detección de límites, prioridades y otros vehículos.
- **Diseño de fitness:** una función de aptitud mal calibrada puede premiar estrategias no deseadas, como detenerse indefinidamente, avanzar demasiado despacio o compensar errores graves con comportamientos parciales.
- **Generalización por puntos de aparición:** el modelo debe funcionar en diferentes zonas del mapa, no únicamente en los puntos de entrenamiento más sencillos.
- **Coste computacional:** entrenar decenas de vehículos simultáneamente en Unreal Engine implica carga de CPU, GPU y física.
- **Trazabilidad:** al tratarse de un sistema experimental, es imprescindible registrar métricas suficientes para explicar por qué una iteración mejora o empeora.

Estos retos aparecen de forma combinada durante el desarrollo. Una mejora en sensores puede modificar el fitness, un cambio en intersecciones puede alterar la tasa de colisiones y una reducción de coste computacional puede afectar al número de generaciones evaluables. Por ello, el proyecto no se aborda como una única implementación cerrada, sino como un proceso de ajuste continuo apoyado en métricas y revisión experimental.

Capítulo 3

Tecnologías y herramientas

3.1 UNREAL ENGINE 5

Unreal Engine 5 es el motor sobre el que se desarrolla el simulador. Proporciona renderizado 3D, editor visual, sistema de actores, colisiones, físicas, Blueprints y herramientas de depuración [3]. Su elección viene heredada de la versión inicial del proyecto y resulta coherente con los requisitos de realismo visual, interacción en tiempo real y despliegue del simulador como aplicación ejecutable.

El trabajo se implementa principalmente mediante Blueprints, lo que permite integrar lógica de IA, controladores, sensores y eventos de tráfico dentro del propio editor. Este enfoque facilita la iteración rápida, aunque introduce retos de mantenibilidad cuando los grafos crecen. Por ello se han documentado los Blueprints principales del gestor evolutivo, el controlador de IA, el vehículo autónomo y los actores de señalización. Las denominaciones concretas de estos módulos se recogen en el anexo de fuentes para mantener la trazabilidad técnica sin sobrecargar el texto principal.

3.2 SISTEMA DE FÍSICAS CHAOS VEHICLE

El vehículo se controla mediante el sistema de físicas Chaos Vehicles de Unreal Engine [4]. La red neuronal no mueve directamente el actor, sino que genera consignas continuas que se aplican al componente de movimiento del vehículo. La salida de dirección se envía como *steering input*; la salida de aceleración/frenado se interpreta de forma separada: valores positivos se aplican como acelerador y valores negativos como freno.

Esta separación es importante porque mantiene el comportamiento dentro de la física del motor. El agente aprende a actuar sobre el vehículo, pero la respuesta final depende de velocidad, fricción, inercia, contacto con el suelo y restricciones propias del modelo físico.

3.3 RAYCASTING Y PERCEPCIÓN LOCAL

La percepción del vehículo se implementa mediante trazas o rayos simulados. El controlador lanza once sensores en un ángulo de visión de 180 grados desde el vehículo. Cada sensor devuelve una distancia normalizada al primer obstáculo relevante, con valor cercano a 1 cuando no se detecta un obstáculo próximo y valores menores cuando existe un límite, pared, vehículo o borde de intersección.

Además de los sensores laterales y frontales, se calculan variables derivadas como la distancia frontal a obstáculo, la distancia del sensor izquierdo, la distancia del sensor derecho y la diferencia horizontal entre ambos. Estas magnitudes permiten construir una señal de centrado en carril y detectar correcciones equivocadas.

3.4 RED NEURONAL

La red neuronal empleada es un perceptrón multicapa ligero. Su vector de entrada se compone de:

- velocidad longitudinal normalizada;
- velocidad angular normalizada;
- once lecturas de sensores;
- estado de semáforo activo;
- estado numérico del semáforo;
- distancia normalizada a la línea de parada;
- indicador de parada forzada;
- codificación *one-hot* del tipo de control de tráfico: semáforo, stop, ceda o ninguno;
- sesgo constante.

La configuración actual produce 22 entradas. La capa oculta contiene 16 neuronas y utiliza $\tanh$ como función de activación. La capa de salida contiene dos neuronas, también con $\tanh$, de modo que sus valores se encuentran en el rango $[-1, 1]$. La primera salida controla dirección y la segunda controla aceleración o frenado. La rutina de inferencia no fija internamente esas 22 entradas: deduce la dimensión esperada a partir de la longitud de la matriz de pesos de entrada y del número de neuronas ocultas. Esta guarda permite detectar de forma coherente modelos incompatibles si la configuración cambia.

3.5 ALGORITMOS EVOLUTIVOS

El entrenamiento se basa en población, selección y mutación. El gestor evolutivo crea una población de vehículos, les asigna pesos iniciales o mutados, ejecuta la simulación y registra el fitness final de cada individuo. Los mejores pesos se guardan en disco como modelos reutilizables, distinguiendo entre el mejor modelo histórico y el mejor de sesión.

La tasa de mutación documentada en la configuración actual se sitúa en torno al 4 % para mutaciones ordinarias, combinando variaciones pequeñas y mutaciones grandes excepcionales. Esta estrategia busca mantener diversidad sin destruir completamente comportamientos ya útiles.

3.6 ANÁLISIS DE DATOS

El proyecto genera tres ficheros principales:

- `Fitness_Debug.csv`, con métricas por coche: fitness, tiempo de vida, punto de aparición, razón de muerte, penalizaciones, bonus, datos de conducción, validación de stops y cedas, mutaciones y familia genealógica.
- `Training_Summary.csv`, con resumen por generación de entrenamiento.
- `Test_Summary.csv`, con resumen por generación de test, incluyendo aciertos, errores y tasas.

El análisis posterior se realiza mediante Python, NumPy, Matplotlib y Seaborn. El script `analyse_current.py` valida la calidad de los CSV, detecta automáticamente si la sesión es de entrenamiento o test, genera informes textuales y produce gráficas de evolución, muertes, distribución de fitness, ranking por spawn, control de entradas, convergencia, diversidad, correlaciones y diagnósticos específicos de paradas o cedas. El script `clean_unreal_log.py` permite limpiar logs de Unreal Engine conservando errores, avisos relevantes, mensajes de Blueprint y trazas propias del proyecto.

3.7 CONTROL DE VERSIONES

El trabajo se ha desarrollado con Git/GitLab en un contexto colaborativo. La documentación de seguimiento recoge tareas de limpieza de repositorio, configuración de `.gitignore`, eliminación de binarios pesados y organización de issues y milestones. Esta disciplina ha sido relevante porque el proyecto combina assets de Unreal, Blueprints, datos generados y modelos guardados.

Capítulo 4

Gestión del proyecto

4.1 METODOLOGÍA

El desarrollo se ha organizado mediante un enfoque iterativo e incremental. Las reuniones de seguimiento se celebraron de forma periódica, generalmente cada dos jueves, con participación de tutores, equipo técnico y personas vinculadas al proyecto original. En estas reuniones se revisaban avances, problemas, decisiones de arquitectura y siguientes objetivos. Esta dinámica se aproxima a una metodología ágil por sprints, aunque adaptada al contexto académico y a la disponibilidad del equipo.

La planificación inicial contemplaba una progresión amplia: semáforos y señales, IA vehicular, stops y cedas, perfiles de conductor, peatones, bicicletas, animales, diversidad de assets y optimización. El desarrollo real concentró el esfuerzo en construir una base sólida de IA vehicular, debido a la dificultad técnica de lograr navegación autónoma estable en la ciudad.

4.2 CRONOLOGÍA DE DECISIONES

La tabla 4.1 sintetiza los principales hitos recogidos en las actas de seguimiento.

Tabla 4.1: Principales hitos y decisiones documentadas en reuniones. Fuente: elaboración propia a partir de las actas de seguimiento.

Fecha	Hito o decisión
27/11/2025	Se revisa la integración de semáforos dinámicos y se fija como objetivo la IA vehicular. Se descarta una malla dinámica para reconocer caminos por los problemas previsibles de desarrollo.
11/12/2025	Se descarta el mapeo completo con splines como opción principal y se considera Path Finding de Unreal. Se decide que el modelo analice entorno y contexto, y que Unreal aplique físicamente la acción.

Tabla 4.1: Principales hitos y decisiones documentadas en reuniones (continuación).

Fecha	Hito o decisión
17/12/2025	Se revisan perfiles de conductor y se fijan provisionalmente dos salidas del modelo: rotación y velocidad, ambas en rango $[-1, 1]$.
08/01/2026	Se descarta Path Finding para vehículos y se valida la idea de entrenamiento evolutivo por prueba-error. El objetivo prioritario pasa a ser aprender básicos de conducción.
05/02/2026	Se discuten problemas en curvas e intersecciones. Se acepta temporalmente el etiquetado dinámico de bordes y se plantea la posibilidad de coordinar más de un modelo.
19/02/2026	Se fija como prioridad una demo técnica en dos semanas que demuestre red neuronal, sensores y bordes funcionando por todo el mapa.
05/03/2026	La demostración se clasifica como fallida y no concluyente. Se acuerda analizar exhaustivamente la función de fitness y sesgar escenarios complejos para facilitar aprendizaje.
19/03/2026	La nueva demostración se considera aceptable: el modelo recorre casi la totalidad de la mayoría de la ciudad y cerca de un 50 % sobrevive más de 100 segundos. Se planifica reorganización en GitLab, build Windows y siguientes mejoras.

4.3 ALCANCE

El alcance funcional completado se centra en IA vehicular autónoma. Incluye:

- controlador de IA capaz de aplicar dirección, aceleración y frenado sobre el vehículo físico;
- sensores por raycasting y entradas normalizadas para la red neuronal;
- entrenamiento evolutivo con población, mutación, elitismo y persistencia de modelos;
- integración con semáforos, señales de stop y ceda el paso;
- cálculo de fitness con componentes de avance, centrado, velocidad, cumplimiento normativo, penalizaciones y bonus;
- exportación de métricas y análisis automatizado de resultados;
- separación de puntos de aparición válidos para entrenamiento y test.

Quedan fuera del alcance de esta versión preliminar la implementación completa de peatones, bicicletas, animales y perfiles de conductor totalmente diferenciados. Estos elementos se mantienen como líneas futuras porque dependen de una base vehicular estable.

4.4 GESTIÓN DE RIESGOS

La tabla 4.2 relaciona los riesgos más relevantes con su impacto, probabilidad y estrategia de mitigación.

Tabla 4.2: Riesgos principales del proyecto. Fuente: elaboración propia.

Riesgo	Impacto	Prob.	Mitigación
Rigidez de navegación determinista	Comportamiento poco realista y baja utilidad clínica	Alta	Pivotar a percepción local con sensores y red neuronal.
Fitness mal calibrado	Aprendizaje de estrategias no deseadas	Alta	Instrumentar componentes, exportar CSV y analizar penalizaciones por familia de muerte.
Coste computacional	Entrenamientos lentos y dependencia de GPU/CPU	Media	Poblaciones moderadas, análisis offline y propuesta futura de modo sin renderizado.
Complejidad de intersecciones	Colisiones y violaciones normativas en cruces	Alta	Bordes dinámicos, zonas de cruce, validación de stops/cedas y separación train/test.
Sobreajuste a spawns concretos	Buen rendimiento local pero mala generalización	Media	Normalización por punto de aparición y test con spawns diferenciados.
Pérdida de conocimiento entrenado	Reinicio de aprendizaje entre sesiones	Media	Persistencia de modelos SuperCarModel y SessionCarModel.

4.5 ESTIMACIÓN DE RECURSOS Y PRESUPUESTO

El trabajo se desarrolló en el marco de prácticas y TFG, por lo que no existe una contratación externa directa asociada. No obstante, desde el punto de vista de proyecto técnico, los recursos empleados son:

- equipo de desarrollo con capacidad para Unreal Engine 5;
- licencia y entorno de Unreal Engine;
- repositorio Git/GitLab;
- herramientas Python de análisis;
- tiempo de desarrollo, documentación, reuniones y experimentación.

La estimación económica debe cerrarse con los criterios del centro y del tutor antes de la entrega final. Como base para completar el presupuesto, se propone separar coste humano, coste hardware amortizado, coste software y coste de infraestructura. En este proyecto, el coste software directo es reducido al apoyarse en herramientas sin coste de licencia para el desarrollo académico, mientras que el coste principal corresponde a horas de ingeniería y experimentación.

Capítulo 5

Núcleo del trabajo

5.1 DESCRIPCIÓN GENERAL DE LA SOLUCIÓN

La solución actual se estructura como un sistema de agentes autónomos entrenados dentro del simulador. Cada vehículo dispone de un controlador de IA que percibe el entorno, ejecuta una red neuronal, aplica acciones al sistema físico y registra métricas de comportamiento. Por encima de los vehículos existe un gestor evolutivo que crea poblaciones, asigna pesos, evalúa resultados, selecciona los mejores individuos y guarda modelos.

El sistema se completa con elementos de tráfico que comunican contexto a los controladores: semáforos, señales de stop, señales de ceda y zonas de cruce. Estos elementos no son meros objetos visuales, sino actores con lógica propia que activan zonas de detección, informan al vehículo de líneas de parada, controlan estados y aplican penalizaciones o bonus según el comportamiento observado.

5.2 SUBSISTEMAS

5.2.1 Gestor evolutivo

`BP_EvolutionManager` coordina entrenamiento y test. Sus responsabilidades principales son:

- cargar modelos guardados desde `SuperCarModel` o inicializar pesos si no existen;
- refrescar los puntos de aparición válidos según el modo de entrenamiento o test;
- generar poblaciones de vehículos;
- asignar cerebros aleatorios, cargados o mutados;
- detectar fin de generación cuando todos los vehículos han muerto o finalizado;
- ordenar por fitness, normalizar por spawn y seleccionar el mejor individuo;
- guardar pesos de sesión e histórico;
- exportar resúmenes de entrenamiento y test.

Una decisión relevante es la normalización por punto de aparición. La ciudad contiene zonas de dificultad desigual; comparar directamente fitness de spawns distintos puede favorecer rutas sencillas. Por ello se mantiene el mejor fitness observado por spawn y se selecciona el mejor coche para pesos atendiendo a una relación normalizada respecto a ese punto.

5.2.2 Controlador de IA y red neuronal

BP_AiController actúa como intermediario entre red neuronal y vehículo físico. En cada tick comprueba que el vehículo controlado existe y no está muerto, prepara las entradas de red, ejecuta la inferencia y aplica las salidas al componente de movimiento.

Para evitar saltos bruscos, el controlador dispone de variables de suavizado como MaxSteeringRate y MaxThrottleRate. La versión documentada mantiene un seguimiento estable entre objetivo y entrada aplicada, con gap medio cero en los informes finales, lo que indica que la señal de control se está transmitiendo sin divergencias instrumentales.

La inferencia se calcula manualmente sobre arrays de pesos:

$$h_j = \tanh \left(\sum_i w_{j,i}^{IH} x_i \right) \quad (5.1)$$

$$y_k = \tanh \left(\sum_j w_{k,j}^{HO} h_j \right) \quad (5.2)$$

La ecuación 5.1 calcula las activaciones ocultas y la ecuación 5.2 obtiene las dos salidas de control, donde x_i son las entradas normalizadas, h_j las neuronas ocultas e y_k las salidas. El sistema incluye guardas para detectar cerebros inválidos: número incorrecto de entradas, pesos ausentes o dimensiones incoherentes. La dimensión de entrada esperada se calcula como la longitud de los pesos de entrada dividida por el número de neuronas ocultas, en lugar de compararse con una constante. Si el error se repite cinco veces, el coche se elimina con razón de muerte INVALID_BRAIN.

5.2.3 Percepción mediante sensores

El controlador lanza once trazas en abanico sobre 180 grados. Cada sensor devuelve una distancia normalizada hasta el primer elemento relevante. Los bordes de intersección se tratan con lógica específica: un borde virtual solo interviene en la lectura cuando su intención de dirección y su trigger coinciden con la decisión vigente del vehículo. Los bordes asociados a otras trayectorias se ignoran, de modo que una misma intersección puede contener límites sensoriales distintos para cada maniobra.

La decisión se modela mediante la enumeración `E_DirectionIntent`, con hasta cuatro alternativas de salida. Al entrar en un `BP_IntersectionTrigger`, el vehículo selecciona aleatoriamente una de las opciones habilitadas para el modo de entrenamiento o de test. El controlador conserva tanto la dirección elegida como el trigger que la originó. Cada `BP_IntersectionBorders` declara la pareja dirección-trigger a la que pertenece, lo que permite que los rayos perciban únicamente los límites que delimitan la trayectoria seleccionada.

Las entradas sensoriales se combinan con información de tráfico. Si el vehículo entra en la zona de un semáforo, stop o ceda, el actor correspondiente comunica al controlador el tipo de control, la línea de parada y la distancia de entrada. Esta mezcla de percepción geométrica y contexto normativo permite que la red no dependa únicamente de distancia a obstáculos.

5.2.4 Sistema de fitness

La función de fitness se ha convertido en uno de los elementos centrales del proyecto. Su objetivo no es solo premiar distancia o velocidad, sino guiar comportamientos compatibles con conducción urbana. La versión documentada actualmente incluye:

- recompensa por avance, velocidad útil y centrado respecto a sensores laterales;
- recompensa por mantener una relación coherente entre velocidad y distancia frontal;
- penalización por exceso de velocidad ante obstáculos;
- penalización por corregir en sentido contrario cuando el vehículo está descentrado;
- penalización por marcha atrás, estancamiento, salida de calzada y colisiones;
- penalización por violar semáforos, stops, cedas y zonas no navegables;
- bonus por completar correctamente stops y cedas;
- métricas acumuladas de frenado, aceleración, espera, steering y contexto de parada.

La instrumentación permite explicar por qué muere cada coche y qué componentes dominan su fitness. Esto fue decisivo tras la demostración fallida del 5 de marzo de 2026, cuando se acordó analizar el fitness con detalle para detectar cálculos o restricciones incorrectas.

5.2.5 Semáforos

El sistema de semáforos se compone de un controlador de intersección y de actores de semáforo. El controlador alterna fases norte-sur y este-oeste, incluyendo verde, ámbar y todo rojo. Cada semáforo mantiene materiales dinámicos para representar estado visual y una zona de sensor que detecta vehículos.

Cuando un vehículo entra en la zona, el semáforo comunica estado, línea de parada y distancia. Si el semáforo está en rojo o ámbar restrictivo, el controlador activa contexto de parada. Al salir de la zona, se comprueba si el vehículo atravesó la línea a una velocidad legal. El sistema aprende umbrales de velocidad legal a partir de muestras previas, evitando depender siempre de un valor fijo.

5.2.6 Stop y ceda el paso

Las señales BP_Sign_Stop y BP_Sign_Yield amplían la interacción normativa. En stop se exige detención suficiente dentro de una ventana dinámica antes de la línea, espera mínima y comprobación de cruce libre. Si se cumple, se aplica bonus y se libera el vehículo; si no, se aplica penalización y muerte por STOP_VIOLATION. En ceda se monitoriza velocidad, aproximación y disponibilidad de la zona de cruce, con penalización por YIELD_VIOLATION y bonus al completar correctamente.

La diferencia entre stop y ceda es metodológicamente importante. El stop requiere detención explícita; el ceda permite continuar si la velocidad y la prioridad son compatibles con la seguridad del cruce.

5.3 PIPELINE EXPERIMENTAL

Cada ejecución genera registros en CSV. El análisis posterior produce informes y gráficas. Este pipeline permite comparar sesiones, detectar regresiones y localizar spawns problemáticos. En las últimas sesiones se emplearon métricas como:

- *BestFit* máximo y medio;
- *MeanFit* por generación;
- tiempo medio de vida;
- porcentaje de fitness medio negativo;
- razones de muerte por familia;
- fallos legales;
- bonus de stop y ceda;
- distribución por puntos de aparición;
- correlaciones entre componentes y fitness final;
- estabilidad de steering y throttle.

El resultado es un ciclo de desarrollo cerrado: implementar, entrenar, exportar, analizar, diagnosticar y volver a ajustar.

Capítulo 6

Resultados

6.1 DATOS ANALIZADOS

La evaluación de referencia utiliza la sesión de entrenamiento iniciada el 8 de junio de 2026 a las 12:16, con 2067 generaciones y 66144 registros de vehículos. Para test no se selecciona una única ejecución favorable: se agregan las seis sesiones realizadas ese mismo día entre las 12:41 y las 13:43. En conjunto contienen 30 ejecuciones, 1560 evaluaciones y 30 observaciones de cada uno de los 52 puntos de aparición.

Todos los informes automáticos del bloque principal califican la calidad de los datos como correcta y no muestran alertas fuertes de desalineación entre Summary y Debug. Las sesiones de test cargan modelos persistidos, tienen la mutación desactivada y mantienen el mismo tamaño de población, por lo que se analizan como repeticiones bajo una configuración común. Para contextualizar su evolución, también se agregan los ocho tests disponibles del 2 de junio y los ocho del 5 de junio.

6.2 RESUMEN CUANTITATIVO

Tabla 6.1: Resultados principales del entrenamiento y del bloque de test del 8 de junio de 2026. Fuente: elaboración propia a partir de los CSV del proyecto.

Métrica	Entrenamiento	Test agregado
Generaciones o ejecuciones	2067	30
Registros de vehículos	66144	1560
<i>BestFit</i> máximo	112447.60	85530.55
<i>MeanFit</i> medio	16162.51	31222.36
<i>MeanFit</i> mediano	16007.06	31176.54
Tiempo medio de vida	45.10 s	53.13 s
Ejecuciones con <i>MeanFit</i> negativo	0.00 %	0.00 %
Colisiones	69.89 %	74.62 %
Fallos legales	0.29 %	0.06 %
LAZY	0.00 %	0.00 %
Tasa de acierto	—	25.19 %

La tabla 6.1 confirma la ausencia de fitness medio negativo y de muertes LAZY, así como una incidencia legal baja. No obstante, las colisiones constituyen aproximadamente siete de cada diez finalizaciones tanto durante el aprendizaje como en test. El mayor *MeanFit* de test respecto al entrenamiento no implica que el modelo mejore durante la evaluación: responde a que ambos modos utilizan poblaciones, condiciones de ejecución y objetivos de medición diferentes.

6.3 EVOLUCIÓN DEL ENTRENAMIENTO

El mejor fitness de la sesión se alcanza en la generación 117, con 112447.60, mientras que el fitness medio máximo aparece en la generación 36, con 29470.20. La media de *MeanFit* es 16162.51 y ninguna generación presenta un valor medio negativo.

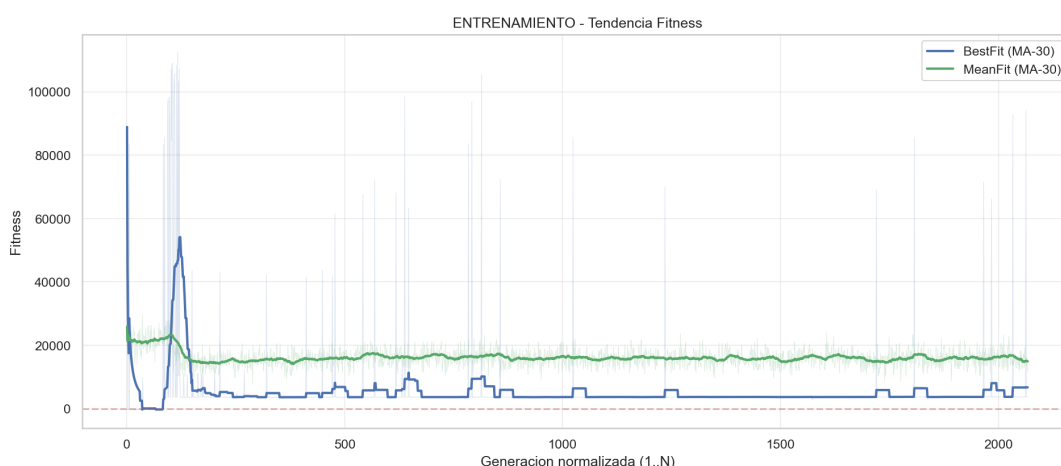


Figura 6.1: Tendencia de fitness y tiempo medio en el entrenamiento del 8 de junio de 2026. Fuente: análisis generado a partir de los CSV del proyecto.

La figura 6.1 obliga a matizar esos máximos. Entre las primeras y las últimas 50 generaciones, el fitness medio desciende un 27.5 %, de 21239.35 a 15406.23, mientras el tiempo medio de vida aumenta un 26.8 %, de 36.35 a 46.10 segundos. El informe detecta una tendencia reciente positiva, pero no convergencia clara. Por tanto, el entrenamiento prolonga la supervivencia y mantiene estabilidad frente a penalizaciones extremas, aunque no produce una mejora monótona de la función de aptitud.

La causa LAZY desaparece por completo en los 66144 registros. Los fallos legales bajan al 0.29 %, frente al 6.82 % de la sesión de referencia del 2 de junio. Sin embargo, el analizador identifica 8691 posibles atascos ante stop entre 19697 finalizaciones por tiempo, un 44.12 %. Se trata de una heurística y no de 8691 infracciones confirmadas, pero su magnitud aconseja revisar los umbrales de detección y realizar tests dirigidos sobre esos puntos.

6.4 EVALUACIÓN DE TEST

En las seis sesiones del 8 de junio se registran 393 aciertos de 1560 evaluaciones, equivalentes al 25.19 %. El intervalo de confianza de Wilson al 95 % se sitúa entre el 23.10 % y el 27.41 %. Las tasas por sesión oscilan entre el 20.38 % y el 30.00 %, con una mediana del 25.58 %; esta dispersión confirma que una sola sesión de 260 vehículos puede sobreestimar o infraestimar el rendimiento.

Para comparar fechas se recalculan todos los indicadores directamente desde `Fitness_Debug.csv`. Este criterio evita depender de los totales de `Summary`: el bloque del 2 de junio contiene una fila debug menos de la declarada y una sesión del 5 de junio conserva una generación duplicada en su resumen. Estas incidencias no aparecen en el bloque del 8 de junio. El fitness final medio de la tabla 6.2 se obtiene promediando `Fitness_Final` por vehículo; es, por tanto, un indicador distinto del *MeanFit* calculado en los resúmenes por generación.

Tabla 6.2: Evolución de los bloques de test disponibles en junio de 2026. Fuente: elaboración propia a partir de `Fitness_Debug.csv`.

Fecha	Evaluaciones	Acierto	Fitness final medio	Vida media
2 de junio	1975	16.51 %	27195.46	46.16 s
5 de junio	2080	18.51 %	26599.02	44.98 s
8 de junio	1560	25.19 %	31295.04	53.13 s

Respecto al bloque del 5 de junio, el resultado más reciente mejora 6.68 puntos porcentuales en acierto, un 17.7 % en fitness medio y un 18.1 % en tiempo de vida. Frente al bloque del 2 de junio, la mejora de acierto es de 8.69 puntos. Esta comparación es más robusta que la tasa del 30 % anteriormente documentada, que procedía de una sola sesión de 260 evaluaciones.

6.5 RAZONES DE FINALIZACIÓN

De las 1560 evaluaciones, 1164 terminan en colisión (74.62 %) y 393 por tiempo (25.19 %). También se observan dos vuelcos (0.13 %) y una violación normativa (0.06 %). Los focos de colisión más frecuentes son `NavModifierVolume19`, con 163 casos; `NavModifierVolume53`, con 124; y `BP_IntersectionBorder203`, con 117. Representan respectivamente el 10.45 %, 7.95 % y 7.50 % del total de evaluaciones.

La concentración en actores y bordes concretos indica que la principal limitación ya no es un fallo general de movimiento, sino la robustez geométrica en determinadas aproximaciones e intersecciones. Esta evidencia permite priorizar la inspección de esos elementos del mapa, los sensores locales y la cobertura de sus spawns asociados.

6.6 COMPORTAMIENTO ANTE SEÑALIZACIÓN

En las seis sesiones, el uso de freno dentro de contextos de parada se mantiene entre el 99.33 % y el 99.50 %, mientras el uso global de freno se sitúa entre el 17.55 % y el 19.31 %. Además, el bloque completo solo registra una finalización clasificada como violación normativa. Estos datos respaldan que el contexto de semáforos, stops y cedas modifica de forma efectiva la acción del controlador.

Los informes detectan cinco candidatos fuertes de atasco en stop y dos en ceda entre las 393 finalizaciones por tiempo. Son pocos en test, pero no deben equipararse automáticamente con una conducción correcta: alcanzar el límite temporal puede representar tanto supervivencia útil como inmovilidad prolongada. La siguiente validación deberá separar explícitamente ambos resultados mediante distancia recorrida, progresión de ruta y tiempo detenido.

6.7 DIFICULTAD POR PUNTOS DE APARICIÓN

Cada spawn dispone de 30 observaciones en el bloque más reciente. Los mejores valores medios corresponden a los puntos 9 (61384.0), 26 (60802.3) y 25 (60615.0). En el extremo opuesto aparecen los puntos 5 (2048.0), 39 (5509.5), 23 (5732.5) y 2 (5920.7). En los CSV, estos identificadores siguen el patrón BP_SpawnPointN.

La diferencia entre extremos supera los 59000 puntos de fitness medio. Por ello, la tasa agregada debe acompañarse siempre de resultados por spawn: un modelo puede mejorar globalmente y conservar fallos sistemáticos en una parte del mapa. Los puntos 5, 39, 23 y 2 son candidatos prioritarios para entrenamiento dirigido y revisión de geometría, mientras los puntos de mejor rendimiento pueden servir como control al modificar sensores o recompensas.

6.8 CONCLUSIÓN EXPERIMENTAL

La evidencia nueva refuerza la validez funcional del sistema: el test agregado mejora los tres indicadores principales respecto a los bloques del 2 y 5 de junio, elimina LAZY y mantiene una incidencia normativa muy baja. Al mismo tiempo, evita presentar como definitiva una ejecución aislada favorable. La tasa de acierto del 25.19 % y el predominio de las colisiones muestran que el controlador aún no es suficientemente robusto para su uso como tráfico autónomo final en sesiones clínicas. Las siguientes iteraciones deben centrarse en intersecciones concretas, spawns de bajo rendimiento, discriminación entre supervivencia y atasco, y validaciones repetidas con el mismo protocolo.

Capítulo 7

Normativa y accesibilidad

7.1 PROTECCIÓN DE DATOS

El proyecto se orienta a una aplicación potencial en rehabilitación cognitiva, por lo que debe contemplar desde el diseño la protección de datos personales y, especialmente, de datos de salud. Durante el desarrollo descrito en esta memoria no se ha trabajado con datos reales de pacientes. Los registros analizados corresponden a vehículos autónomos simulados, métricas de entrenamiento y resultados técnicos.

En una futura validación clínica, cualquier uso con personas usuarias deberá ajustarse al Reglamento General de Protección de Datos y a la Ley Orgánica 3/2018 de Protección de Datos Personales y garantía de los derechos digitales [5, 6]. Será necesario definir responsable del tratamiento, finalidad, base jurídica, minimización de datos, conservación, consentimiento informado cuando proceda, medidas de seudonimización y control de acceso. También deberá diferenciarse entre datos necesarios para la evaluación terapéutica y métricas técnicas internas del simulador.

7.2 SEGURIDAD Y USO RESPONSABLE

El simulador no sustituye por sí mismo una evaluación clínica ni una autorización administrativa para conducir. Su función debe entenderse como apoyo a profesionales, entrenamiento controlado y herramienta de observación. Cualquier conclusión sobre capacidad real de conducción debería depender de especialistas y procedimientos establecidos.

Desde el punto de vista técnico, el sistema autónomo genera tráfico artificial. Por tanto, sus fallos no suponen riesgo físico directo durante el entrenamiento virtual, pero sí pueden afectar a la validez de la experiencia. Es importante que las sesiones con pacientes utilicen modelos suficientemente probados para evitar comportamientos incoherentes que confundan al usuario o introduzcan estímulos no deseados.

7.3 ACCESIBILIDAD

El proyecto se dirige a personas con posibles alteraciones cognitivas y físicas. Por ello, la accesibilidad debe contemplarse en varias capas:

- interfaces claras, con instrucciones sencillas y ausencia de sobrecarga visual;
- compatibilidad con periféricos adaptados, como volante, pedales o mandos específicos;
- configuración de dificultad, densidad de tráfico y tipos de eventos;
- posibilidad de repetir escenarios concretos para entrenamiento progresivo;
- exportación de resultados comprensible para profesionales;
- control de mareo o fatiga mediante rendimiento estable y duración ajustable.

El trabajo actual contribuye especialmente a la configuración de dificultad mediante tráfico autónomo y eventos normativos. A futuro, la accesibilidad debe integrarse con perfiles de paciente, parámetros de escenario y supervisión profesional.

Capítulo 8

Conclusiones y líneas futuras

8.1 CONCLUSIONES

El trabajo realizado demuestra la viabilidad de integrar movilidad autónoma basada en redes neuronales y algoritmos evolutivos dentro de un simulador urbano en Unreal Engine 5. La solución actual supera las limitaciones iniciales de navegación determinista y proporciona una arquitectura capaz de percibir el entorno, actuar sobre un vehículo físico, aprender mediante generaciones y producir datos analizables.

La principal aportación técnica es la combinación de cuatro piezas: percepción local por raycasting, red neuronal ligera, gestor evolutivo con persistencia de modelos y función de fitness instrumentada. Esta combinación permite entrenar agentes sin disponer de un dataset previo de conducción humana y, al mismo tiempo, analizar con detalle por qué cada iteración mejora o falla.

También se ha demostrado la importancia de la trazabilidad. Las decisiones más relevantes del proyecto no surgieron de una única implementación, sino de iteraciones documentadas: descarte de mallas dinámicas, prueba y abandono de Path Finding como solución principal, introducción de bordes dinámicos, reformulación del fitness tras una demo fallida y separación entre entrenamiento y test.

Los resultados disponibles son prometedores pero no definitivos. La sesión de entrenamiento del 8 de junio evita generaciones con fitness medio negativo, erradica el estancamiento tipo LAZY y reduce los fallos legales, aunque no muestra convergencia clara y conserva posibles atascos ante señales. Las seis sesiones de test asociadas suman 1560 evaluaciones y alcanzan un 25.19 % de acierto, con un intervalo de confianza del 95 % entre el 23.10 % y el 27.41 %. Este bloque mejora los resultados agregados de los días 2 y 5 de junio, pero las colisiones continúan siendo la familia de finalización dominante. Por ello, la IA todavía requiere mejoras antes de considerarse tráfico autónomo plenamente robusto para sesiones clínicas.

8.2 APORTACIONES REALIZADAS

Las aportaciones principales son:

- diseño de una arquitectura de IA vehicular cuya configuración actual utiliza 22 entradas y dos salidas continuas;
- sensores por raycasting adaptados a límites, obstáculos e intersecciones;
- selección de trayectoria mediante intenciones de dirección, triggers diferenciados para entrenamiento y test, y bordes virtuales asociados a cada maniobra;
- entrenamiento evolutivo con población, mutación, elitismo, pesos cargados y pesos de sesión;
- normalización de fitness por punto de aparición;
- integración de semáforos, stops y cedas con contexto de parada y validación;
- función de fitness con componentes de avance, centrado, velocidad, señales, penalizaciones y bonus;
- persistencia de modelos entrenados;
- pipeline de análisis con informes, gráficas y comparación entre sesiones;
- documentación cronológica de decisiones técnicas.

Estas aportaciones no deben entenderse solo como funcionalidades aisladas, sino como una infraestructura de experimentación. El proyecto deja conectados simulación, entrenamiento, persistencia y análisis, de manera que cada nueva iteración puede evaluarse con criterios comparables y no únicamente mediante observación visual dentro del editor.

8.3 PROBLEMAS ENCONTRADOS

Los problemas más relevantes fueron:

- curva de aprendizaje de Unreal Engine 5 y del sistema Chaos Vehicle;
- rigidez de soluciones basadas en NavMesh para vehículos físicos;
- complejidad de intersecciones, cruces y bordes de calzada;
- dificultad para diseñar un fitness que premie conducción útil sin favorecer atajos;
- coste computacional de entrenar poblaciones con renderizado y física activa;
- necesidad de depurar comportamientos emergentes no previstos, como estancamiento, marcha atrás o correcciones incorrectas.

La resolución de estos problemas exigió combinar depuración técnica y análisis de datos. En varias fases, el comportamiento observado en pantalla no era suficiente para explicar los fallos, por lo que fue necesario revisar métricas exportadas, razones de muerte, penalizaciones acumuladas y diferencias entre puntos de aparición.

8.4 LÍNEAS FUTURAS

Las líneas futuras recomendadas son:

- ampliar el entrenamiento específico en spawns problemáticos, especialmente aquellos con colisiones recurrentes;
- mejorar la percepción en intersecciones y la semántica de bordes;
- implementar perfiles de conductor normal, inseguro y agresivo;
- añadir interacción entre vehículos y reglas de prioridad más completas;
- extender el sistema a peatones, bicicletas y otros agentes autónomos;
- crear un modo de entrenamiento sin renderizado o desacoplado para acelerar generaciones;
- mantener bloques de test repetidos y comparables para estimar el acierto con confianza estadística;
- preparar un protocolo de validación con profesionales de ASPAYM antes de su uso con pacientes.

Agradecimientos

Pendiente de redacción definitiva.

Bibliografía

- [1] L. Fuente, *CognitiveCarSimulatorReloaded - IA*, propuesta de proyecto, 2025.
- [2] E. Romero Yuste, *CognitiveCarSimulator*, Trabajo de Fin de Grado, Grado en Desarrollo de Aplicaciones Interactivas 3D y Videojuegos, Escuela Politécnica Superior de Zamora, 2025.
- [3] Epic Games, *Unreal Engine 5 Documentation*, Epic Developer Community. Disponible en: documentación oficial de Unreal Engine (consulta: 8 de junio de 2026).
- [4] Epic Games, *Chaos Vehicles*, Epic Developer Community. Disponible en: documentación oficial de Chaos Vehicles (consulta: 8 de junio de 2026).
- [5] Parlamento Europeo y Consejo de la Unión Europea, *Reglamento (UE) 2016/679, de 27 de abril de 2016, relativo a la protección de las personas físicas en lo que respecta al tratamiento de datos personales y a la libre circulación de estos datos*, Diario Oficial de la Unión Europea, 2016. Disponible en: EUR-Lex (consulta: 8 de junio de 2026).
- [6] Jefatura del Estado, *Ley Orgánica 3/2018, de 5 de diciembre, de Protección de Datos Personales y garantía de los derechos digitales*, Boletín Oficial del Estado, núm. 294, 6 de diciembre de 2018. Disponible en: BOE (consulta: 8 de junio de 2026).

Apéndice A

Fuentes analizadas

A.1 DOCUMENTACIÓN ADMINISTRATIVA Y DE FORMATO

Para la redacción de esta memoria se han revisado los documentos incluidos en INFO/UTILES. Entre ellos destacan:

- portada y normas básicas de estilo de la Escuela de Ingenierías Industrial, Informática y Aeroespacial;
- solicitud de preinscripción del TFG, con título, descripción y tutoría;
- resolución favorable de preinscripción de fecha 18 de mayo de 2026;
- propuesta *CognitiveCarSimulatorReloaded - IA*;
- memoria previa *CognitiveCarSimulator*;
- informe de seguimiento de prácticas HP SCDS;
- memoria final de prácticas del alumno;
- estructura orientativa propuesta por el tutor.

A.2 ACTAS DE REUNIÓN

Se han analizado ocho actas de seguimiento, comprendidas entre el 27/11/2025 y el 19/03/2026. Estas actas permiten justificar las decisiones de arquitectura y los cambios de enfoque realizados durante el proyecto.

A.3 DESCRIPCIONES DEL PROGRAMA

Se han revisado descripciones fechadas de los módulos, incluyendo las actualizaciones de red neuronal e intersecciones del 5 de junio de 2026:

- EVOLUTIONMANAGER, con el gestor de generaciones, selección, mutación, persistencia y test;
- REDNEURONAL, con entradas, arquitectura de red, sensores, inicialización, mutación e inferencia;
- FITNESS, con cálculo de aptitud, penalizaciones, bonus y exportación de métricas;
- INTERSECTIONS, con semáforos, stops, cedas, zonas de cruce y validación normativa;
- OTROS, con macros auxiliares, raytracing, detección de estancamiento y backlog.

A.4 RESULTADOS EXPERIMENTALES

Los resultados documentados proceden de la carpeta INFO/ANALISIS_DATOS/ Analisis. Para esta primera redacción se han tomado como referencia principal:

- train_2026-06-08_12-16, con 2067 generaciones y 66144 registros;
- seis sesiones de test del 8 de junio, realizadas entre las 12:41 y las 13:43, con 30 ejecuciones y 1560 registros en total;
- los ocho tests del 2 de junio y los ocho del 5 de junio, con 1975 y 2080 registros debug respectivamente, empleados como bloques de comparación;
- train_2026-06-02_12-00 y train_2026-06-05_11-47, empleados para contextualizar la evolución del entrenamiento.

Los informes automáticos de las sesiones principales indican calidad de datos correcta y ausencia de alertas fuertes entre los CSV Summary y Debug. La figura del capítulo de resultados procede del script de análisis del proyecto. El texto identifica la sesión representada y diferencia sus métricas de los agregados de test.